EN3160 - Assignment 03: Neural Networks

Name: Ahamed A.M.S
Index number: 230020R
GitHub profile: https://github.com/shaahidahamed

In [3]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

import random
import numpy as np
from pathlib import Path
import zipfile
import os
import urllib.request
from torchvision import datasets
from torch.utils.data import DataLoader
from torchvision import models
from torchvision.models import ResNet18_Weights

SEED = 42
random.seed(SEED)
```

In [4]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('PyTorch:', torch.__version__)
print('Torchvision:', torchvision.__version__)
print('Device:', device)
```

```
PyTorch: 2.13.0+cpu
Torchvision: 0.28.0+cpu
Device: cpu
```

Question 1: Neural Network with One Hidden Layer

Listing 1 shows the code for a single dense-layer network with manually computed forward propagation and backpropagation. Make the following changes:

(a) Add a hidden layer with 100 neurons and a sigmoid activation function.

(b) Replace the existing loss function with cross-entropy loss (refer to Slide 102).

(c) Train the network for 10 epochs and report the training and test accuracies.

In [5]:
```
# 1. Dataloading
transform = transforms.Compose([
```

```
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

batch_size = 50
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                          shuffle=True, num_workers=2)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                         shuffle=False, num_workers=2)
classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', '
print(len(trainset), 'training images;', len(testset), 'test images')
```

```
50000 training images; 10000 test images
```

In [6]:

```
# Custom activation and loss functions
def sigmoid(x):
    return 1 / (1 + torch.exp(-x))

def cross_entropy_loss(y_pred, y_true):
    return -torch.sum(y_true * torch.log(y_pred + 1e-9)) / y_true.shape[0]

# 2. Define Network Parameters
Din = 3 * 32 * 32  # Input size (flattened CIFAR-10 image size)
H = 100            # Hidden layer size
K = 10             # Output size (number of classes in CIFAR-10)
std = 1e-5

# Initialize weights and biases
w1 = torch.randn(Din, H) * std
b1 = torch.zeros(H)
w2 = torch.randn(H, K) * std
b2 = torch.zeros(K)

# Hyperparameters
iterations = 10
lr = 1e-3          # Learning rate
lr_decay = 0.9     # Learning rate decay
reg = 0            # Regularization
loss_history = []


# 3. Training Loop
for t in range(iterations):
    running_loss = 0.0

    for i, data in enumerate(trainloader, 0):
        # Get inputs and labels
        inputs, labels = data
        Ntr = inputs.shape[0]
        x_train = inputs.view(Ntr, -1)
        y_train_onehot = nn.functional.one_hot(labels, K).float()
```

```
        # Forward pass
        h = sigmoid(x_train.mm(w1) + b1)
        y_pred = h.mm(w2) + b2
        y_pred_softmax = torch.softmax(y_pred, dim=1)

        # Loss calculation (Cross-Entropy with regularization)
        loss = cross_entropy_loss(y_pred_softmax, y_train_onehot) + reg * (torch.su
        loss_history.append(loss.item())
        running_loss += loss.item()

        # Backpropagation
        dy_pred = (y_pred_softmax - y_train_onehot)

        dw2 = h.t().mm(dy_pred) + reg * w2
        db2 = dy_pred.sum(dim=0)

        dh = dy_pred.mm(w2.t())
        dz = dh * h * (1 - h)

        dw1 = x_train.t().mm(dz) + reg * w1
        db1 = dz.sum(dim=0)

        # Parameter update
        w1 -= lr * dw1
        b1 -= lr * db1
        w2 -= lr * dw2
        b2 -= lr * db2

    # Print loss for every epoch
    if t % 1 == 0:
        print(f"Epoch {t + 1}/{iterations}, Loss: {running_loss / len(trainloader)}

    # Learning rate decay
    lr *= lr_decay


# 4. Plotting the Loss History
plt.plot(loss_history)
plt.title("Loss History")
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.show()


# 5. Calculate Accuracy on Training Set
correct_train = 0
total_train = 0

with torch.no_grad():
    for data in trainloader:
        inputs, labels = data
        Ntr = inputs.shape[0]
        x_train = inputs.view(Ntr, -1)

        # Forward pass
        h = sigmoid(x_train.mm(w1) + b1)
```

```
        y_train_pred = h.mm(w2) + b2

        predicted_train = torch.argmax(y_train_pred, dim=1)
        total_train += labels.size(0)
        correct_train += (predicted_train == labels).sum().item()

train_acc = 100 * correct_train / total_train
print(f"Training accuracy: {train_acc:.2f}%")


# 6. Calculate Accuracy on Test Set
correct_test = 0
total_test = 0

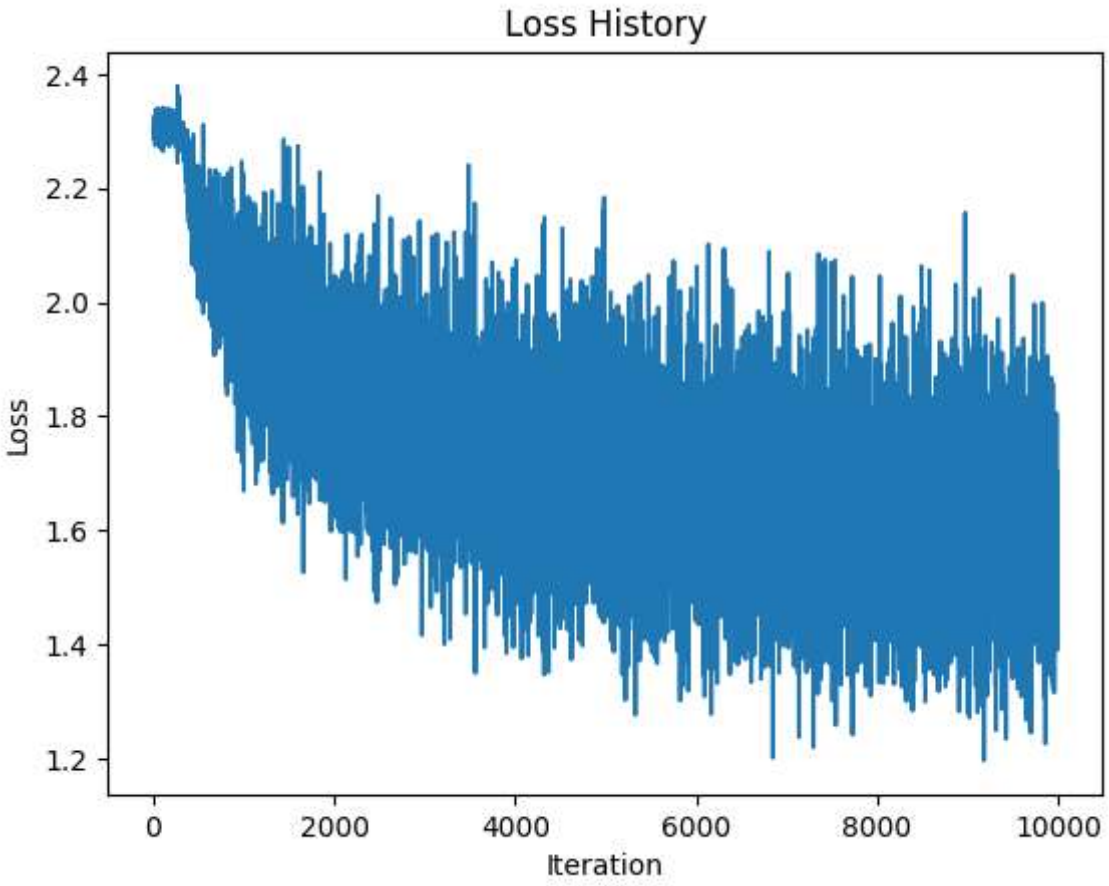
with torch.no_grad():
    for data in testloader:
        inputs, labels = data
        Nte = inputs.shape[0]
        x_test = inputs.view(Nte, -1)

        # Forward pass
        h = sigmoid(x_test.mm(w1) + b1)
        y_test_pred = h.mm(w2) + b2

        predicted_test = torch.argmax(y_test_pred, dim=1)
        total_test += labels.size(0)
        correct_test += (predicted_test == labels).sum().item()

test_acc = 100 * correct_test / total_test
print(f"Test accuracy: {test_acc:.2f}%")
```

```
Epoch 1/10, Loss: 2.1550494223833083
Epoch 2/10, Loss: 1.9005253471136092
Epoch 3/10, Loss: 1.8052645877599716
Epoch 4/10, Loss: 1.7496716455221175
Epoch 5/10, Loss: 1.709389125943184
Epoch 6/10, Loss: 1.6801190789937972
Epoch 7/10, Loss: 1.6575498154163362
Epoch 8/10, Loss: 1.6383537381887436
Epoch 9/10, Loss: 1.622553701043129
Epoch 10/10, Loss: 1.6082558854818345
```




```
Training accuracy: 44.71%
Test accuracy: 43.55%
```

Question 2: CNN Models for MNIST Classification

(a) Create a LeNet-5 network for the MNIST dataset using PyTorch. Report the training and test accuracies after 10 epochs.

(b) Implement a modern small CNN with three convolutional layers, batch normalization, and dropout. Train it under the same conditions as LeNet-5.

(c) For both models, record and plot the training and test accuracy and loss for each epoch. Compute and display a confusion matrix on the test set for each model.

In [7]:
```
mnist_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

mnist_trainset = datasets.MNIST(root='./data', train=True, download=True, transform
mnist_testset = datasets.MNIST(root='./data', train=False, download=True, transform

mnist_trainloader = DataLoader(mnist_trainset, batch_size=64, shuffle=True,num_work
mnist_testloader = DataLoader(mnist_testset, batch_size=64, shuffle=False,num_worke

print(len(mnist_trainset), 'training images;', len(mnist_testset), 'test images')
```

```
60000 training images; 10000 test images
```

In [8]:
```python
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5)    # 28 -> 24
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)   # 12 -> 8
        self.fc1 = nn.Linear(16 * 4 * 4, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = torch.max_pool2d(x, 2)

        x = torch.relu(self.conv2(x))
        x = torch.max_pool2d(x, 2)

        x = torch.flatten(x, 1)
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)

lenet = LeNet5().to(device)
```

In [9]:
```python
class ModernSmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)
        self.drop2d = nn.Dropout2d(0.25)
        self.fc1 = nn.Linear(128 * 7 * 7, 128)
        self.drop = nn.Dropout(0.5)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = torch.relu(self.bn1(self.conv1(x)))
        x = torch.relu(self.bn2(self.conv2(x)))
        x = nn.MaxPool2d(2, 2)(x)       # 28 -> 14
        x = self.drop2d(x)

        x = torch.relu(self.bn3(self.conv3(x)))
        x = nn.MaxPool2d(2, 2)(x)       # 14 -> 7
        x = self.drop2d(x)

        x = torch.flatten(x, 1)
        x = torch.relu(self.fc1(x))
        x = self.drop(x)
        return self.fc2(x)

modern_cnn = ModernSmallCNN().to(device)
```

In [10]:

```python
# Train the Model
def train_model(model, optimizer, criterion, train_loader, test_loader, num_epochs=
    history = {
        'train_loss': [], 'train_acc': [],
        'test_loss': [], 'test_acc': []
    }

    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            # Forward pass
            outputs = model(images)
            loss = criterion(outputs, labels)

            # Backward pass and optimization
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * labels.size(0)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

        train_loss = running_loss / total
        train_accuracy = 100 * correct / total

        # Test the model after each epoch
        model.eval()
        test_loss = 0.0
        test_correct = 0
        test_total = 0

        with torch.no_grad():
            for images, labels in test_loader:
                images, labels = images.to(device), labels.to(device)

                outputs = model(images)
                loss = criterion(outputs, labels)

                test_loss += loss.item() * labels.size(0)
                _, predicted = torch.max(outputs, 1)
                test_total += labels.size(0)
                test_correct += (predicted == labels).sum().item()

        test_loss /= test_total
        test_accuracy = 100 * test_correct / test_total

        history['train_loss'].append(train_loss)
```

```
        history['train_acc'].append(train_accuracy)
        history['test_loss'].append(test_loss)
        history['test_acc'].append(test_accuracy)

        print(
            f'Epoch [{epoch+1}/{num_epochs}], '
            f'Train Loss: {train_loss:.4f}, '
            f'Train Accuracy: {train_accuracy:.2f}%, '
            f'Test Loss: {test_loss:.4f}, '
            f'Test Accuracy: {test_accuracy:.2f}%'
        )

    return history


# Train LeNet-5
torch.manual_seed(SEED)
lenet = LeNet5().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(lenet.parameters(), lr=0.01, momentum=0.9)

lenet_history = train_model(
    lenet, optimizer, criterion,
    mnist_trainloader, mnist_testloader
)


# Train Modern CNN
torch.manual_seed(SEED)
modern_cnn = ModernSmallCNN().to(device)
optimizer = optim.SGD(modern_cnn.parameters(), lr=0.01, momentum=0.9)

modern_history = train_model(
    modern_cnn, optimizer, criterion,
    mnist_trainloader, mnist_testloader
)


# Final Results
print("\nFinal Results after 10 Epochs")
print(f'LeNet-5 Training Accuracy: {lenet_history["train_acc"][-1]:.2f}%')
print(f'LeNet-5 Test Accuracy: {lenet_history["test_acc"][-1]:.2f}%')
print(f'Modern CNN Training Accuracy: {modern_history["train_acc"][-1]:.2f}%')
print(f'Modern CNN Test Accuracy: {modern_history["test_acc"][-1]:.2f}%')
```

```
Epoch [1/10], Train Loss: 0.3057, Train Accuracy: 90.55%, Test Loss: 0.0617, Test Ac
curacy: 97.95%
Epoch [2/10], Train Loss: 0.0654, Train Accuracy: 97.94%, Test Loss: 0.0460, Test Ac
curacy: 98.67%
Epoch [3/10], Train Loss: 0.0462, Train Accuracy: 98.54%, Test Loss: 0.0378, Test Ac
curacy: 98.85%
Epoch [4/10], Train Loss: 0.0361, Train Accuracy: 98.90%, Test Loss: 0.0381, Test Ac
curacy: 98.89%
Epoch [5/10], Train Loss: 0.0311, Train Accuracy: 99.03%, Test Loss: 0.0352, Test Ac
curacy: 98.94%
Epoch [6/10], Train Loss: 0.0263, Train Accuracy: 99.16%, Test Loss: 0.0416, Test Ac
curacy: 98.71%
Epoch [7/10], Train Loss: 0.0215, Train Accuracy: 99.31%, Test Loss: 0.0353, Test Ac
curacy: 98.90%
Epoch [8/10], Train Loss: 0.0194, Train Accuracy: 99.37%, Test Loss: 0.0370, Test Ac
curacy: 98.83%
Epoch [9/10], Train Loss: 0.0154, Train Accuracy: 99.49%, Test Loss: 0.0412, Test Ac
curacy: 98.81%
Epoch [10/10], Train Loss: 0.0143, Train Accuracy: 99.53%, Test Loss: 0.0411, Test A
ccuracy: 98.80%
Epoch [1/10], Train Loss: 0.2920, Train Accuracy: 90.87%, Test Loss: 0.0580, Test Ac
curacy: 98.18%
Epoch [2/10], Train Loss: 0.1345, Train Accuracy: 96.11%, Test Loss: 0.0448, Test Ac
curacy: 98.61%
Epoch [3/10], Train Loss: 0.1029, Train Accuracy: 96.99%, Test Loss: 0.0357, Test Ac
curacy: 98.86%
Epoch [4/10], Train Loss: 0.0885, Train Accuracy: 97.40%, Test Loss: 0.0321, Test Ac
curacy: 98.98%
Epoch [5/10], Train Loss: 0.0767, Train Accuracy: 97.77%, Test Loss: 0.0305, Test Ac
curacy: 99.10%
Epoch [6/10], Train Loss: 0.0660, Train Accuracy: 98.01%, Test Loss: 0.0231, Test Ac
curacy: 99.26%
Epoch [7/10], Train Loss: 0.0588, Train Accuracy: 98.28%, Test Loss: 0.0242, Test Ac
curacy: 99.17%
Epoch [8/10], Train Loss: 0.0534, Train Accuracy: 98.40%, Test Loss: 0.0236, Test Ac
curacy: 99.28%
Epoch [9/10], Train Loss: 0.0485, Train Accuracy: 98.61%, Test Loss: 0.0224, Test Ac
curacy: 99.33%
Epoch [10/10], Train Loss: 0.0450, Train Accuracy: 98.62%, Test Loss: 0.0226, Test A
ccuracy: 99.28%

Final Results after 10 Epochs
LeNet-5 Training Accuracy: 99.53%
LeNet-5 Test Accuracy: 98.80%
Modern CNN Training Accuracy: 98.62%
Modern CNN Test Accuracy: 99.28%
```

In [11]:

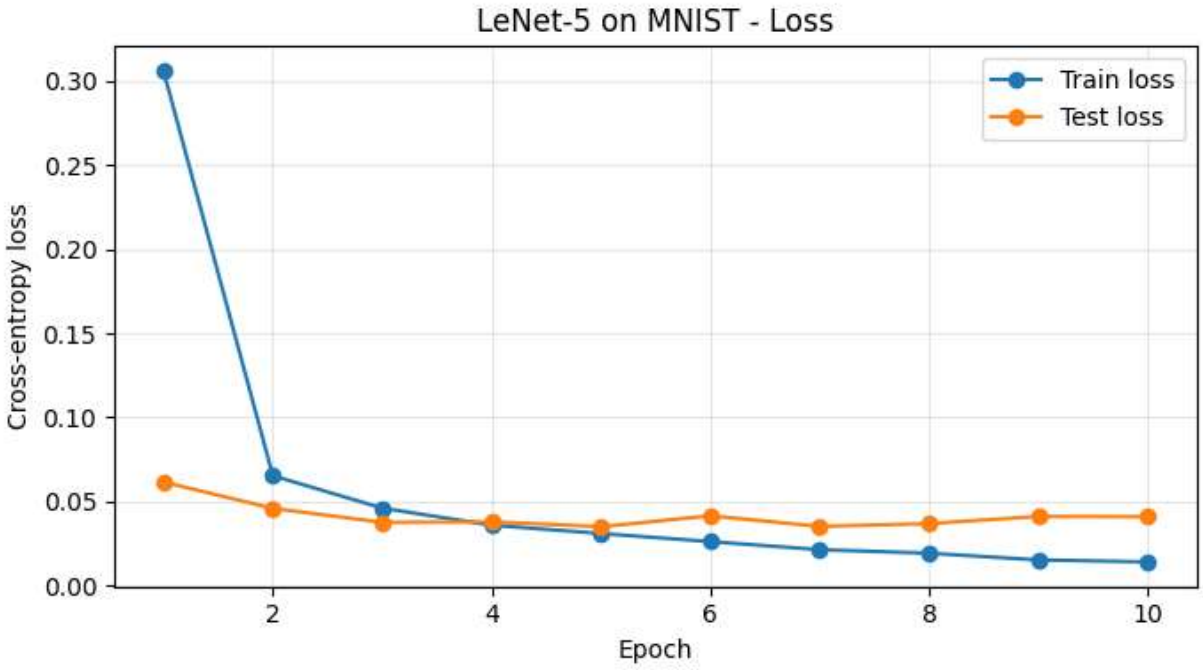
```python
def plot_history(history, model_name, prefix):
    epochs = np.arange(1, len(history['train_loss']) + 1)

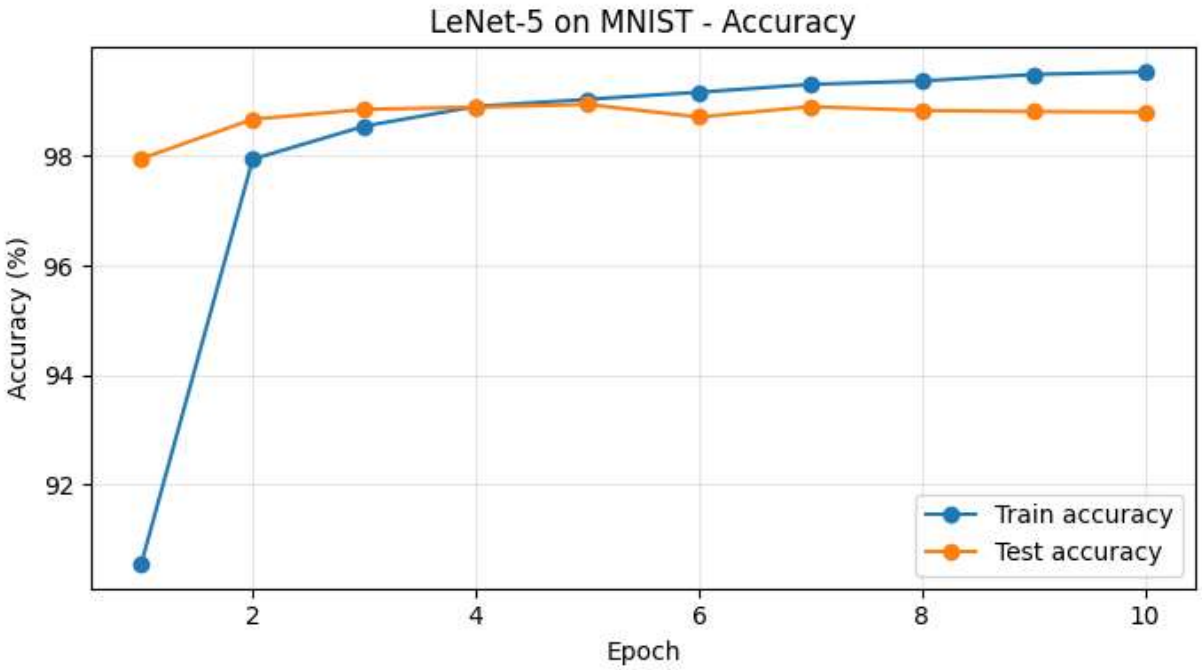
    plt.figure(figsize=(7, 4))
    plt.plot(epochs, history['train_loss'], marker='o', label='Train loss')
    plt.plot(epochs, history['test_loss'], marker='o', label='Test loss')
    plt.xlabel('Epoch')
    plt.ylabel('Cross-entropy loss')
    plt.title(f'{model_name} - Loss')
```

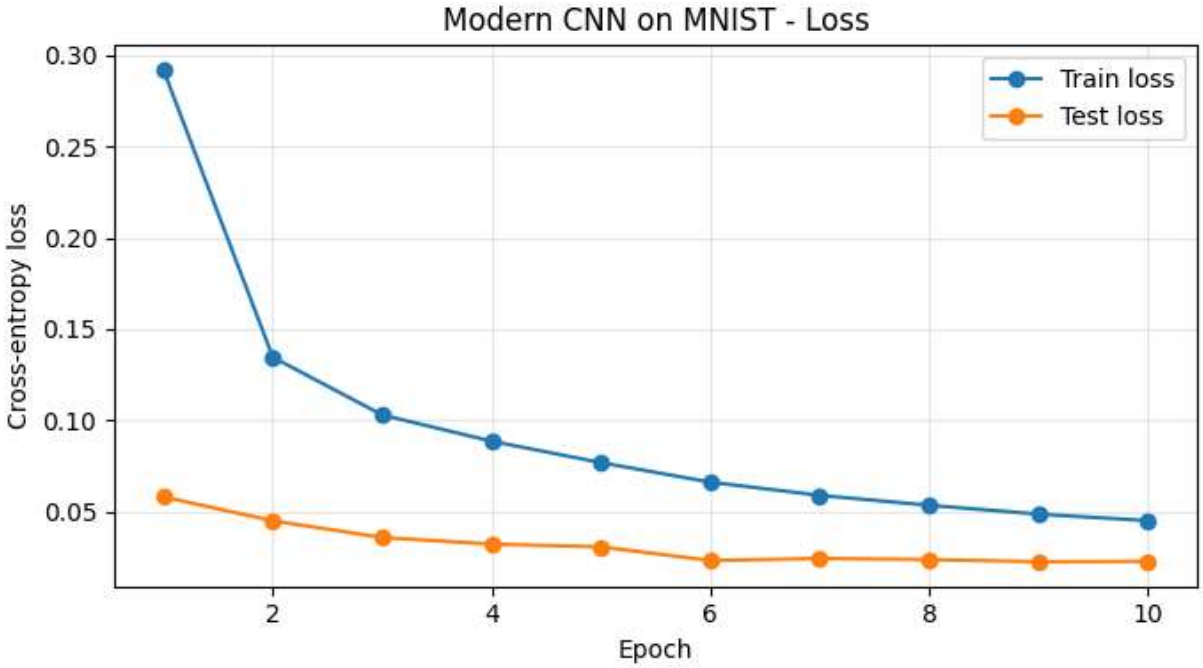
```
    plt.grid(alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()

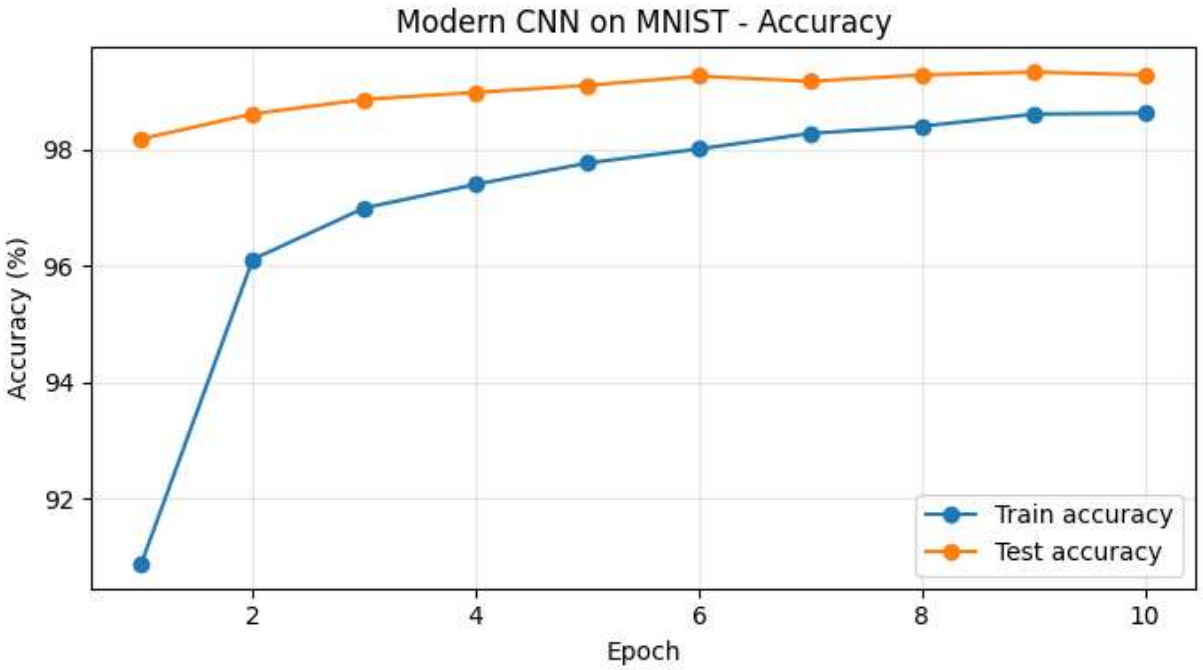
    plt.figure(figsize=(7, 4))
    plt.plot(epochs, history['train_acc'], marker='o', label='Train accuracy')
    plt.plot(epochs, history['test_acc'], marker='o', label='Test accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy (%)')
    plt.title(f'{model_name} - Accuracy')
    plt.grid(alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()

plot_history(lenet_history, 'LeNet-5 on MNIST', 'q2_lenet')
plot_history(modern_history, 'Modern CNN on MNIST', 'q2_modern_cnn')
```




LeNet-5 on MNIST - Accuracy
Accuracy (%)
98
96
94
92
2
4
6
8
10
Epoch
Train accuracy
Test accuracy


Modern CNN on MNIST - Loss
Cross-entropy loss
0.30
0.25
0.20
0.15
0.10
0.05
2
4
6
8
10
Epoch
Train loss
Test loss



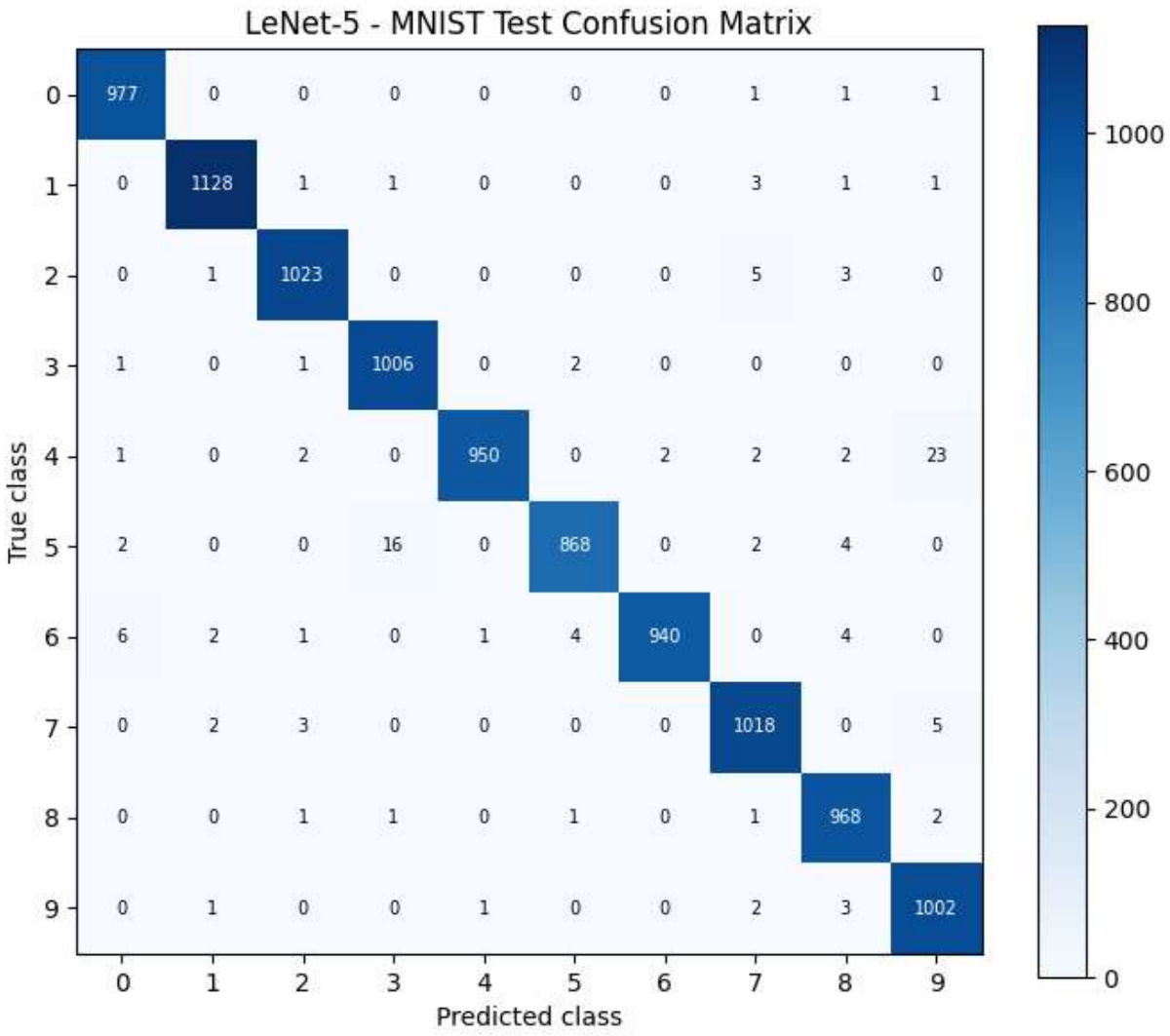
In [12]:

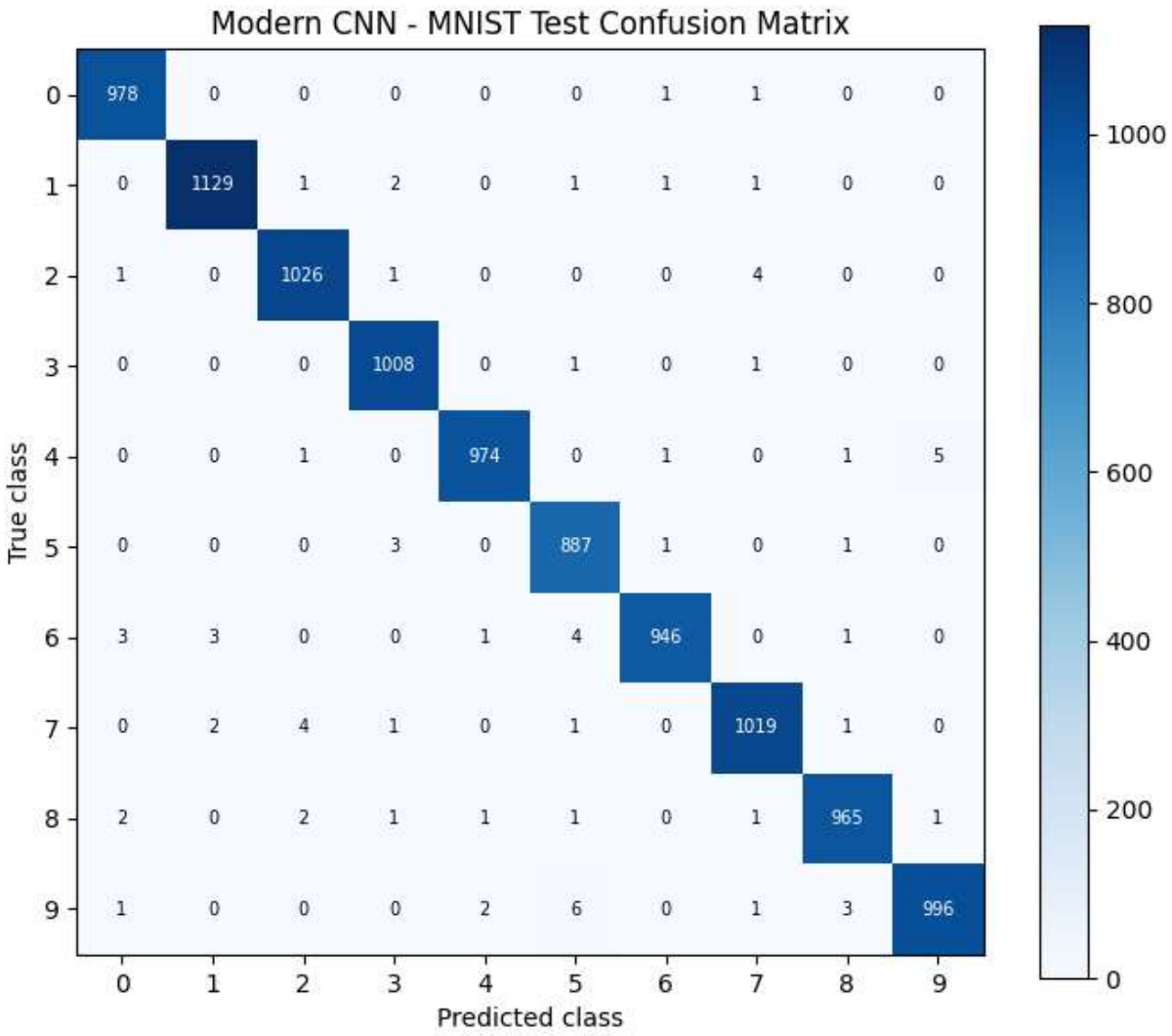
```
def confusion_matrix_tensor(model, loader, num_classes):
    cm = torch.zeros(num_classes, num_classes, dtype=torch.int64)
    model.eval()
    with torch.no_grad():
        for x, y in loader:
            x = x.to(device)
            pred = model(x).argmax(1).cpu()
            y = y.cpu()
            for true, guessed in zip(y, pred):
                cm[true.long(), guessed.long()] += 1
    return cm.numpy()


def plot_confusion_matrix(cm, labels, title, filename):
    plt.figure(figsize=(7, 6))
    plt.imshow(cm, interpolation='nearest', cmap='Blues')
    plt.title(title)
    plt.colorbar()
    ticks = np.arange(len(labels))
    plt.xticks(ticks, labels)
    plt.yticks(ticks, labels)
    plt.xlabel('Predicted class')
    plt.ylabel('True class')
    threshold = cm.max() / 2.0
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(j, i, str(cm[i, j]), ha='center', va='center',
                     color='white' if cm[i, j] > threshold else 'black', fontsize=7
    plt.tight_layout()
    plt.show()

lenet_cm = confusion_matrix_tensor(lenet, mnist_testloader, 10)
modern_cm = confusion_matrix_tensor(modern_cnn, mnist_testloader, 10)
```

```
plot_confusion_matrix(lenet_cm, list(range(10)), 'LeNet-5 - MNIST Test Confusion Ma
plot_confusion_matrix(modern_cm, list(range(10)), 'Modern CNN - MNIST Test Confusio
```





Question 3: Transfer Learning with ResNet-18

Based on the PyTorch tutorial on transfer learning, use the pre-trained ResNet-18 network trained on ImageNet1K to classify the Hymenoptera dataset using the following approaches:

(a) Fine-tuning the pre-trained ResNet-18 network.

(b) Using the pre-trained ResNet-18 network as a fixed feature extractor.

Report the results obtained from both approaches.

In [13]:
```
# Download and extract the dataset
data_dir = "./hymenoptera_data"
url = "https://download.pytorch.org/tutorial/hymenoptera_data.zip"

if not os.path.exists(data_dir):
    print("Downloading Hymenoptera dataset...")
    urllib.request.urlretrieve(url, "hymenoptera_data.zip")

    with zipfile.ZipFile("hymenoptera_data.zip", "r") as zip_ref:
```

```
        zip_ref.extractall(".")

# Define image transformations
data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(
            [0.485, 0.456, 0.406],
            [0.229, 0.224, 0.225]
        )
    ]),
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize(
            [0.485, 0.456, 0.406],
            [0.229, 0.224, 0.225]
        )
    ])
}

# Load training and validation datasets
train_dataset = datasets.ImageFolder(
    root=f"{data_dir}/train",
    transform=data_transforms['train']
)

test_dataset = datasets.ImageFolder(
    root=f"{data_dir}/val",
    transform=data_transforms['val']
)

# Create data loaders
train_loader = DataLoader(
    train_dataset,
    batch_size=4,
    shuffle=True,
    num_workers=2
)

test_loader = DataLoader(
    test_dataset,
    batch_size=4,
    shuffle=False,
    num_workers=2
)

# Dataset information
class_names = train_dataset.classes

print("Classes:", class_names)
print("Training images:", len(train_dataset))
print("Validation images:", len(test_dataset))
```

```
Classes: ['ants', 'bees']
Training images: 244
Validation images: 153
```

In [14]:

```python
# Train and evaluate ResNet-18
def train_transfer_model(model, optimizer, criterion, num_epochs=10, feature_extrac
    history = {
        'train_loss': [], 'train_acc': [],
        'val_loss': [], 'val_acc': []
    }

    for epoch in range(num_epochs):
        model.train()

        # Keep pretrained BatchNorm statistics fixed during feature extraction
        if feature_extraction:
            model.eval()

        running_loss = 0.0
        correct = 0
        total = 0

        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)

            # Forward pass
            outputs = model(inputs)
            loss = criterion(outputs, labels)

            # Backward pass and optimization
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * labels.size(0)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

        train_loss = running_loss / total
        train_accuracy = 100 * correct / total

        # Validation
        model.eval()
        val_loss = 0.0
        val_correct = 0
        val_total = 0

        with torch.no_grad():
            for inputs, labels in test_loader:
                inputs, labels = inputs.to(device), labels.to(device)

                outputs = model(inputs)
                loss = criterion(outputs, labels)

                val_loss += loss.item() * labels.size(0)
```

```
                _, predicted = torch.max(outputs, 1)
                val_total += labels.size(0)
                val_correct += (predicted == labels).sum().item()

        val_loss /= val_total
        val_accuracy = 100 * val_correct / val_total

        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_accuracy)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_accuracy)

        print(
            f'Epoch [{epoch+1}/{num_epochs}], '
            f'Train Loss: {train_loss:.4f}, '
            f'Train Accuracy: {train_accuracy:.2f}%, '
            f'Val Loss: {val_loss:.4f}, '
            f'Val Accuracy: {val_accuracy:.2f}%'
        )

    return history
```

In [15]:

```
# Load pretrained ResNet-18
torch.manual_seed(SEED)
resnet18_ft = torchvision.models.resnet18(
    weights=torchvision.models.ResNet18_Weights.DEFAULT
)

# Replace final layer for 2 classes (ants and bees)
num_features = resnet18_ft.fc.in_features
resnet18_ft.fc = nn.Linear(num_features, 2)
resnet18_ft = resnet18_ft.to(device)

# Loss and optimizer
criterion = nn.CrossEntropyLoss()
optimizer_ft = optim.SGD(
    resnet18_ft.parameters(),
    lr=0.0001,
    momentum=0.9
)

# Train for 10 epochs
finetune_history = train_transfer_model(
    resnet18_ft,
    optimizer_ft,
    criterion,
    num_epochs=10
)

print(f'\nFine-Tuning Final Validation Accuracy: {finetune_history["val_acc"][-1]:.
```

```
Epoch [1/10], Train Loss: 0.6644, Train Accuracy: 60.66%, Val Loss: 0.4804, Val Accu
racy: 83.66%
Epoch [2/10], Train Loss: 0.5435, Train Accuracy: 75.82%, Val Loss: 0.3434, Val Accu
racy: 92.16%
Epoch [3/10], Train Loss: 0.4802, Train Accuracy: 75.41%, Val Loss: 0.2784, Val Accu
racy: 92.81%
Epoch [4/10], Train Loss: 0.4491, Train Accuracy: 80.74%, Val Loss: 0.3012, Val Accu
racy: 87.58%
Epoch [5/10], Train Loss: 0.4754, Train Accuracy: 75.00%, Val Loss: 0.2521, Val Accu
racy: 91.50%
Epoch [6/10], Train Loss: 0.4214, Train Accuracy: 81.15%, Val Loss: 0.2187, Val Accu
racy: 94.12%
Epoch [7/10], Train Loss: 0.3592, Train Accuracy: 83.61%, Val Loss: 0.2013, Val Accu
racy: 94.12%
Epoch [8/10], Train Loss: 0.3479, Train Accuracy: 84.84%, Val Loss: 0.1977, Val Accu
racy: 94.12%
Epoch [9/10], Train Loss: 0.3643, Train Accuracy: 84.02%, Val Loss: 0.1925, Val Accu
racy: 95.42%
Epoch [10/10], Train Loss: 0.3758, Train Accuracy: 83.20%, Val Loss: 0.1974, Val Acc
uracy: 94.12%

Fine-Tuning Final Validation Accuracy: 94.12%
```

In [16]:

```python
# Load pretrained ResNet-18
torch.manual_seed(SEED)
resnet18_fx = torchvision.models.resnet18(
    weights=torchvision.models.ResNet18_Weights.DEFAULT
)

# Freeze pretrained layers
for param in resnet18_fx.parameters():
    param.requires_grad = False

# Replace final layer for 2 classes
num_features = resnet18_fx.fc.in_features
resnet18_fx.fc = nn.Linear(num_features, 2)
resnet18_fx = resnet18_fx.to(device)

# Only train the final classification layer
optimizer_fx = optim.SGD(
    resnet18_fx.fc.parameters(),
    lr=0.001,
    momentum=0.9
)

# Train for 10 epochs
feature_history = train_transfer_model(
    resnet18_fx,
    optimizer_fx,
    criterion,
    num_epochs=10,
    feature_extraction=True
)

print(f'\nFeature Extraction Final Validation Accuracy: {feature_history["val_acc"]
```

```
Epoch [1/10], Train Loss: 0.2999, Train Accuracy: 88.11%, Val Loss: 0.1267, Val Accu
racy: 94.77%
Epoch [2/10], Train Loss: 0.2063, Train Accuracy: 90.98%, Val Loss: 0.1321, Val Accu
racy: 96.08%
Epoch [3/10], Train Loss: 0.1352, Train Accuracy: 93.85%, Val Loss: 0.1358, Val Accu
racy: 96.08%
Epoch [4/10], Train Loss: 0.1319, Train Accuracy: 94.26%, Val Loss: 0.1678, Val Accu
racy: 94.77%
Epoch [5/10], Train Loss: 0.0976, Train Accuracy: 96.72%, Val Loss: 0.1267, Val Accu
racy: 95.42%
Epoch [6/10], Train Loss: 0.0916, Train Accuracy: 95.08%, Val Loss: 0.1526, Val Accu
racy: 94.77%
Epoch [7/10], Train Loss: 0.0761, Train Accuracy: 96.31%, Val Loss: 0.1340, Val Accu
racy: 96.08%
Epoch [8/10], Train Loss: 0.0752, Train Accuracy: 97.13%, Val Loss: 0.1635, Val Accu
racy: 95.42%
Epoch [9/10], Train Loss: 0.0937, Train Accuracy: 96.31%, Val Loss: 0.1750, Val Accu
racy: 95.42%
Epoch [10/10], Train Loss: 0.1125, Train Accuracy: 95.49%, Val Loss: 0.1620, Val Acc
uracy: 96.08%

Feature Extraction Final Validation Accuracy: 96.08%
```

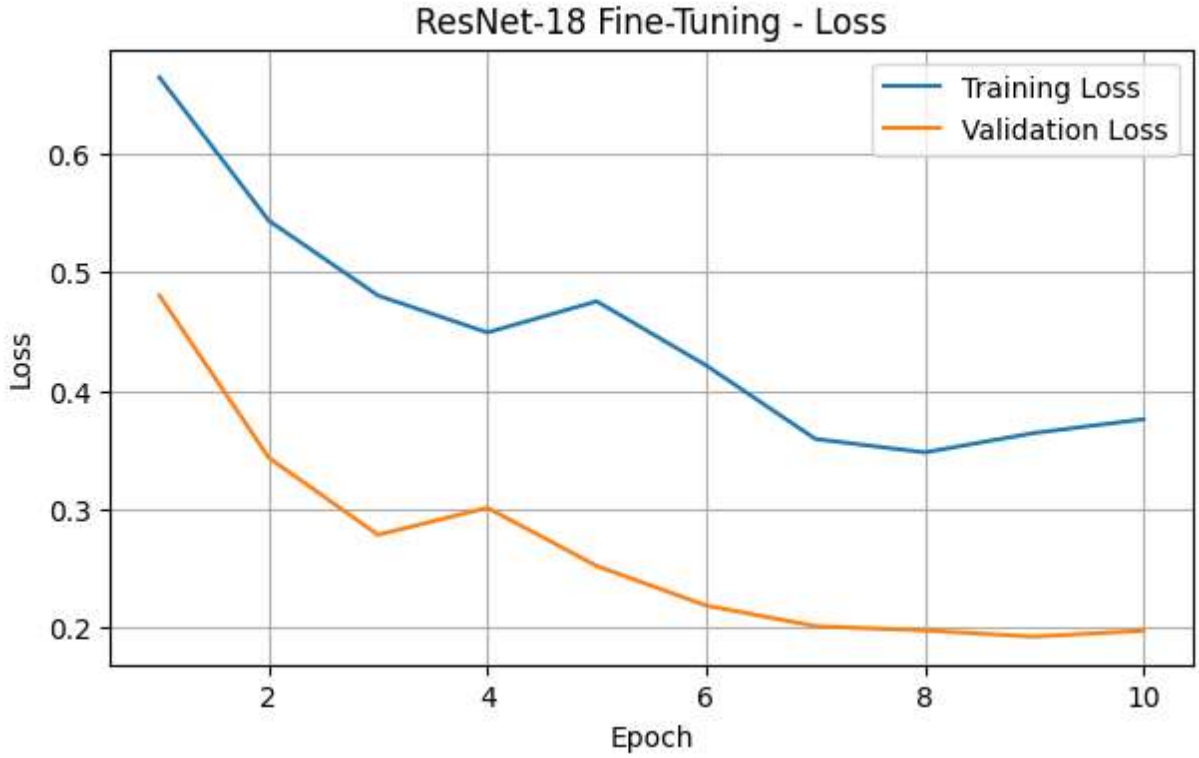
In [17]:

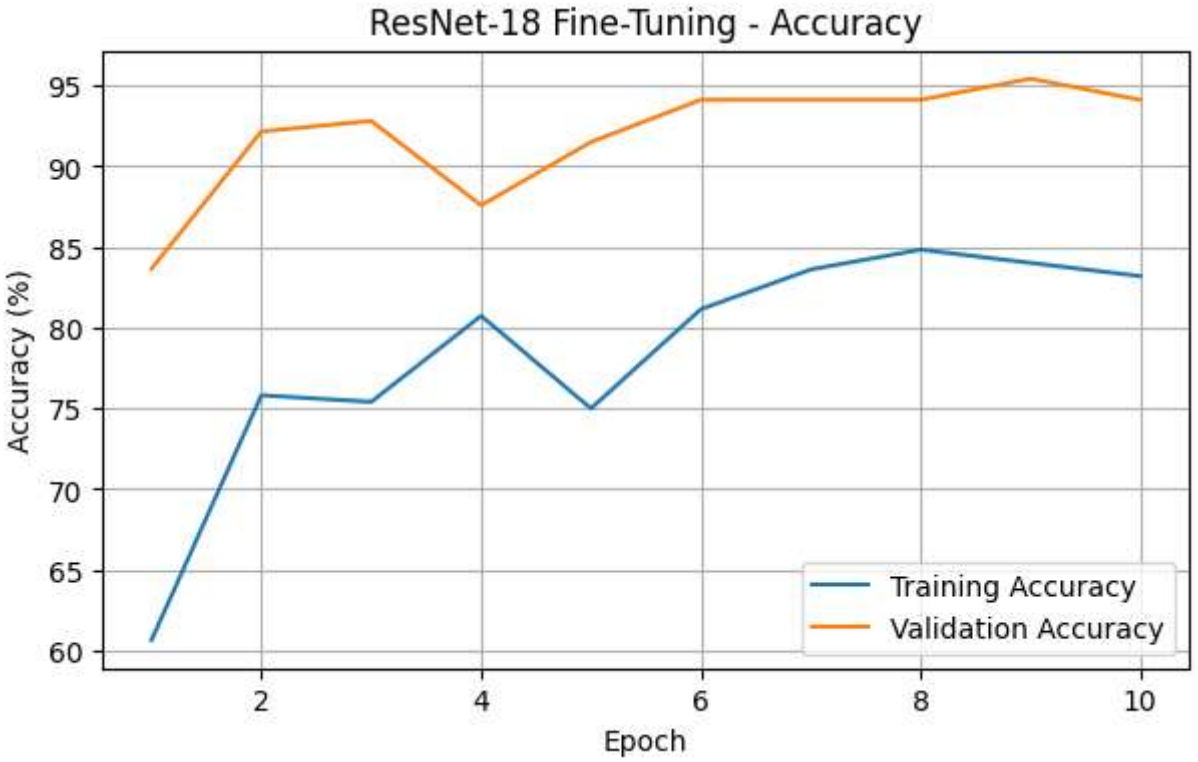
```python
def plot_transfer_history(history, model_name):
    epochs = range(1, len(history['train_loss']) + 1)

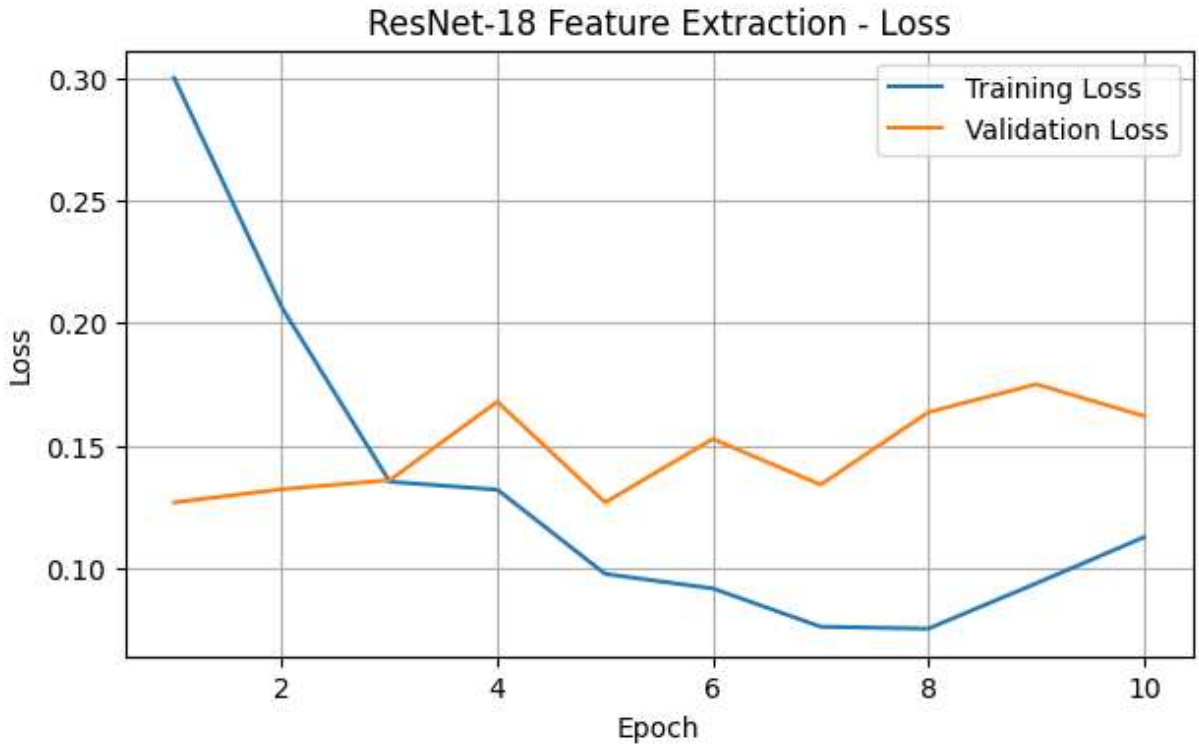
    plt.figure(figsize=(7, 4))
    plt.plot(epochs, history['train_loss'], label='Training Loss')
    plt.plot(epochs, history['val_loss'], label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title(f'{model_name} - Loss')
    plt.legend()
    plt.grid(True)
    plt.show()

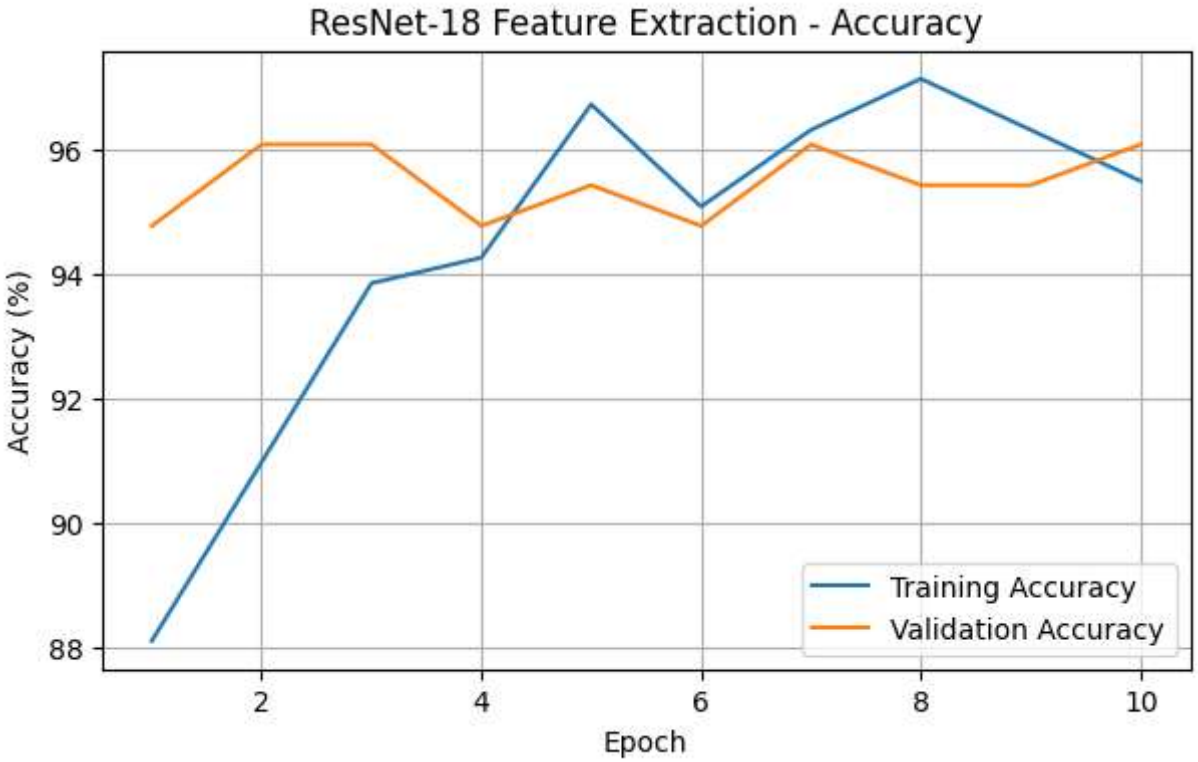
    plt.figure(figsize=(7, 4))
    plt.plot(epochs, history['train_acc'], label='Training Accuracy')
    plt.plot(epochs, history['val_acc'], label='Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy (%)')
    plt.title(f'{model_name} - Accuracy')
    plt.legend()
    plt.grid(True)
    plt.show()


plot_transfer_history(finetune_history, 'ResNet-18 Fine-Tuning')
plot_transfer_history(feature_history, 'ResNet-18 Feature Extraction')
```


ResNet-18 Fine-Tuning - Loss
Training Loss
Validation Loss
Loss
0.6
0.5
0.4
0.3
0.2
2
4
6
8
10
Epoch


ResNet-18 Fine-Tuning - Accuracy
Training Accuracy
Validation Accuracy
Accuracy (%)
95
90
85
80
75
70
65
60
2
4
6
8
10
Epoch





In [18]:
```
print("\nResNet-18 Transfer Learning Results")
print("-" * 65)

print(f'Fine-Tuning:')
print(f'  Training Accuracy: {finetune_history["train_acc"][-1]:.2f}%')
print(f'  Validation Accuracy: {finetune_history["val_acc"][-1]:.2f}%')

print(f'\nFeature Extraction:')
```

```
print(f'  Training Accuracy: {feature_history["train_acc"][-1]:.2f}%')
print(f'  Validation Accuracy: {feature_history["val_acc"][-1]:.2f}%')
```

```
ResNet-18 Transfer Learning Results
----------------------------------------------------------------
Fine-Tuning:
  Training Accuracy: 83.20%
  Validation Accuracy: 94.12%

Feature Extraction:
  Training Accuracy: 95.49%
  Validation Accuracy: 96.08%
```